University of Exeter

ECM2433: The C Family

Lecture 06: Project Structure, Testing, and HashMaps

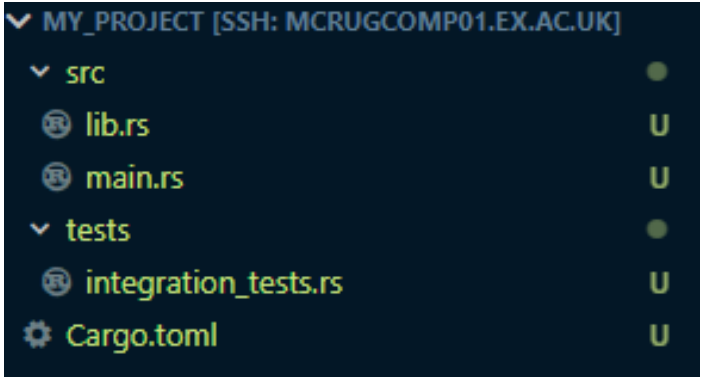
RUST

Why lib.rs?

- So far, all code has lived in `main.rs`
- Problem: you cannot test `main()` automatically
- Solution: move reusable logic into `lib.rs`, keep `main.rs` thin
- `lib.rs`: library crate (reusable, testable)
- `main.rs`: binary crate (entry point, calls library code)
- In C, you separate `.c` files for reuse and separate compilation
- [Book ch 7.1]

University of Exeter

Project layout: binary and library crates



- One project can have both `main.rs` and `lib.rs`
- `main.rs`: compiled into an executable
- `lib.rs`: compiled into a library (used by `main.rs` and tests)
- `tests/`: integration tests
- [Book ch 7.1, 7.5]

University of Exeter

Creating a library with `pub`

```rust
// src/lib.rs
use std::fs;
use std::io;

pub fn count_lines(path: &str) -> Result<usize, io::Error> {
    let contents = fs::read_to_string(path)?;
    let mut count = 0;
    for line in contents.lines() {
        if !line.is_empty() {
            count += 1;
        }
    }
    Ok(count)
}
```

- `pub` makes a function visible outside the module
- Without `pub`, functions are private by default
- This is the `count_lines` function from the previous video, now in `lib.rs`
- [Book ch 7.2]

University of Exeter

Using your library and the `use` keyword

```rust
// src/main.rs
use std::error::Error;
use my_project::count_lines;

fn main() -> Result<(), Box<dyn Error>> {
    let n = count_lines("hello.txt")?;
    println!("{n} non-empty lines");
    Ok(())
}
```

- `use my_project::count_lines`: imports `count_lines` from your library crate
- The crate name matches the `[package] name` in `Cargo.toml`
- `use std::error::Error`: imports from the standard library
- `use` is a path shortcut, not text inclusion like `#include`
- [Book ch 7.2, 7.4]

University of Exeter

Unit tests in lib.rs

```rust
// src/lib.rs
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        // passes if true
        assert!(add(2, 3) > 0);
        // passes if equal
        assert_eq!(add(2, 3), 5);
        // passes if not equal
        assert_ne!(add(2, 3), 0);
    }
}
```

- `#[cfg(test)]`: only compiled when running tests
- `mod tests`: a module that groups test functions
- `use super::*`: imports everything from the parent module (`lib.rs`)
- `#[test]`: marks a function as a test

- `assert!(expr)`: passes if expression is true
- `assert_eq!(a, b)`: passes if equal
- `assert_ne!(a, b)`: passes if not equal

- [Book ch 11.1, 11.3]

Testing functions that return `Result`

```rust
// src/lib.rs
use std::num::ParseIntError;

pub fn int_parse(x: &str) -> Result<i32, ParseIntError> {
    x.parse()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_valid_input() -> Result<(), ParseIntError> {
        let result = int_parse("42")?;
        assert_eq!(result, 42);
        Ok(())
    }

    #[test]
    fn test_invalid_input() {
        assert!(int_parse("hello").is_err());
    }
}
```

- Tests can return `Result`: use `?` instead of `unwrap()`
- A test fails if it panics or returns `Err`
- `.is_err()`: returns `true` if the `Result` is `Err`
- [Book ch 11.1]

Integration tests

```rust
// tests/integration_tests.rs
use my_project::add;
use my_project::int_parse;
use std::num::ParseIntError;

#[test]
fn test_add_from_outside() {
    assert_eq!(add(10, 20), 30);
}

#[test]
fn test_int_parse_from_outside() -> Result<(), ParseIntError> {
    let result = int_parse("99")?;
    assert_eq!(result, 99);
    Ok(())
}
```

- Integration tests live in the `tests/` directory
- Each file is compiled as a separate crate
- Can only access `pub` functions (no `use super::*`)
- No `#[cfg(test)]` or `mod tests` needed
- Tests your library the way external code would
- [Book ch 11.3]

University of Exeter

Running `cargo test` output

```
[gad212@mcrugcomp01 my_project]$ cargo test
   Compiling my_project v0.1.0 (/home/links/gad212/my_project)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.61s
     Running unittests src/lib.rs (target/debug/deps/my_project-962d67288a7ec15a)

running 3 tests
test tests::test_add ... ok
test tests::test_invalid_input ... ok
test tests::test_valid_input ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

     Running unittests src/main.rs (target/debug/deps/my_project-9b3a2721cc36f0c3)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

     Running tests/integration_tests.rs (target/debug/deps/integration_tests-a4b8b333161e08f5)

running 2 tests
test test_add_from_outside ... ok
test test_int_parse_from_outside ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

   Doc-tests my_project

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

- `cargo test`: runs all unit and integration tests
- Unit tests grouped under `src/lib.rs`
- Integration tests grouped by file
- [Book ch 11.2]

HashMap: creating and accessing

```rust
use std::collections::HashMap;

let mut scores: HashMap<char, i32> = HashMap::new();

scores.insert('a', 10);
scores.insert('b', 20);

println!("{}", scores[&'a']);        // 10 (panics if missing)
println!("{:?}", scores.get(&'b')); // Some(20)
println!("{:?}", scores.get(&'c')); // None
```

- `HashMap<K, V>`: stores key-value pairs
- `HashMap::new()`: creates an empty map
- `.insert(key, value)`: adds or overwrites an entry
- `map[&key]`: direct access (panics if key missing)
- `.get(&key)`: returns `Option<&V>` (safe access)
- [Book ch 8.3]

University of Exeter

HashMap: the `entry` API

```rust
use std::collections::HashMap;

let mut char_counts: HashMap<char, i32> = HashMap::new();
let text = "abracadabra";

for ch in text.chars() {
    let count = char_counts.entry(ch).or_insert(0);
    *count += 1;
} // counts: a=5, b=2, r=2, c=1, d=1 (map order may vary)
```

- `.entry(key)` : looks up a key, returns an `Entry`
- `.or_insert(default)` : inserts default if key is missing, returns `&mut V`
- The returned `&mut V` lets you update the value in place
- Common pattern: counting, aggregating totals
- [Book ch 8.3]

University of Exeter

Demo

University of Exeter

Recap

- `lib.rs`: library crate (reusable, testable); `main.rs`: binary crate (entry point)
- `pub`: makes functions visible outside the module
- `use`: brings items into scope from your crate or the standard library
- `#[cfg(test)] mod tests`: unit tests alongside your code
- Integration tests in `tests/` directory: test the public interface
- `HashMap<K, V>`: key-value storage with `insert`, `get`, and the `entry` API
- Required reading:
  - [Book ch 7.1-7.2, 7.5]: Packages, crates, and paths
  - [Book ch 8.3]: HashMaps
  - [Book ch 11.1-11.3]: Testing

University of Exeter